



Humber ASV Document Title

Humber Polytechnic

Version 1.0.0

Date: May 12, 2026

Contents

1	Introduction	5
2	Systems Overview	5
3	Software System	5
3.1	Primary Computer	7
3.1.1	ROS2 Humble	7
4	Software Development	9
4.1	Development Resources	9
4.1.1	Documentation	10
4.1.2	Codebase	10
4.1.3	Communication Channels	11
4.2	Contribution Guidelines	11
4.2.1	Code of Conduct	11
5	Remote Connections	13
5.1	TailScale Connection	13
5.2	RustDesk Connection	13
6	Running the Software	14
6.1	Troubleshooting	14
7	ROS2 Design	16
A	Glossary	17
B	Bibliography	18

List of Figures

1	System Context Diagram	5
2	Level 1 Data Flow Diagram	6
3	Level 1.1 Data Flow Diagram for Primary Computer	7
4	Software Architecture Diagram	8
5	Software and tools required for development.	9
6	Code of Conduct	12

List of Tables

1	Primary Computer Software Stack	7
2	Primary Computer ROS2 Nodes	16
3	Current Primary Computer ROS2 Nodes Data Distribution	16
4	Proposed Computer ROS2 Node Data Distribution	16

1 Introduction

Humber ASV is a multidisciplinary team of passionate Humber Polytechnic students and professionals dedicated to advancing autonomous maritime technology.[2] The team is focused on designing, building, and operating their autonomous surface vehicle (ASV) to contribute to the field of maritime robotics. The Loon-E ASV is one of the key projects undertaken by HumberASV, showcasing the team's commitment to innovation and excellence in autonomous maritime technology.[2] This document serves as a comprehensive guidebook for the Loon-E ASV, providing detailed information on its design, operation, and maintenance. It is intended to be a valuable resource for stakeholders and developers involved in the Loon-E project, offering insights into the ASV's capabilities, specifications, and best practices for its use and upkeep. The on-board computer system includes hardware components such as the Jetson Orin computer, Zedx camera, and various student-lead designs [7].

2 Systems Overview

The Loon-E ASV is an autonomous surface vehicle designed to operate on water. It is not equipped to be underwater; please don't try to submerge it. Loon-E is designed to be modular and adaptable, allowing for various configurations and payloads to suit different tasks as competition requirements evolve [7].

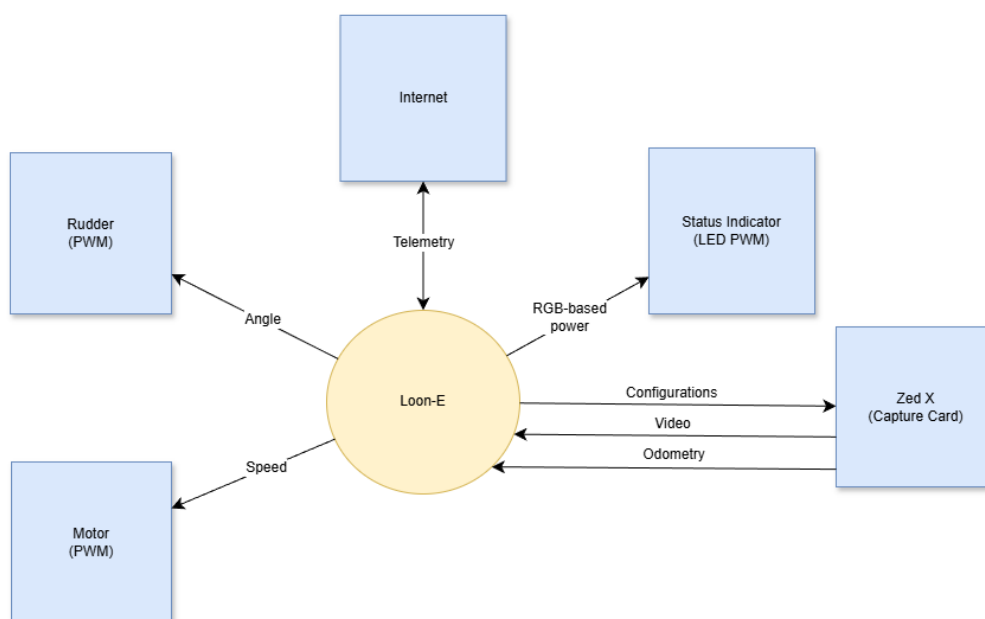


Figure 1: System Context Diagram

3 Software System

The software system of the Loon-E ASV is responsible for controlling the vehicle's operations, including navigation, sensor data processing, and communication. The entire system can be illustrated in Figure 2, which illustrates a 3 process system consisting of a base station, the primary computer, and a web client. These 3 components work together to realize the software system of the ASV.

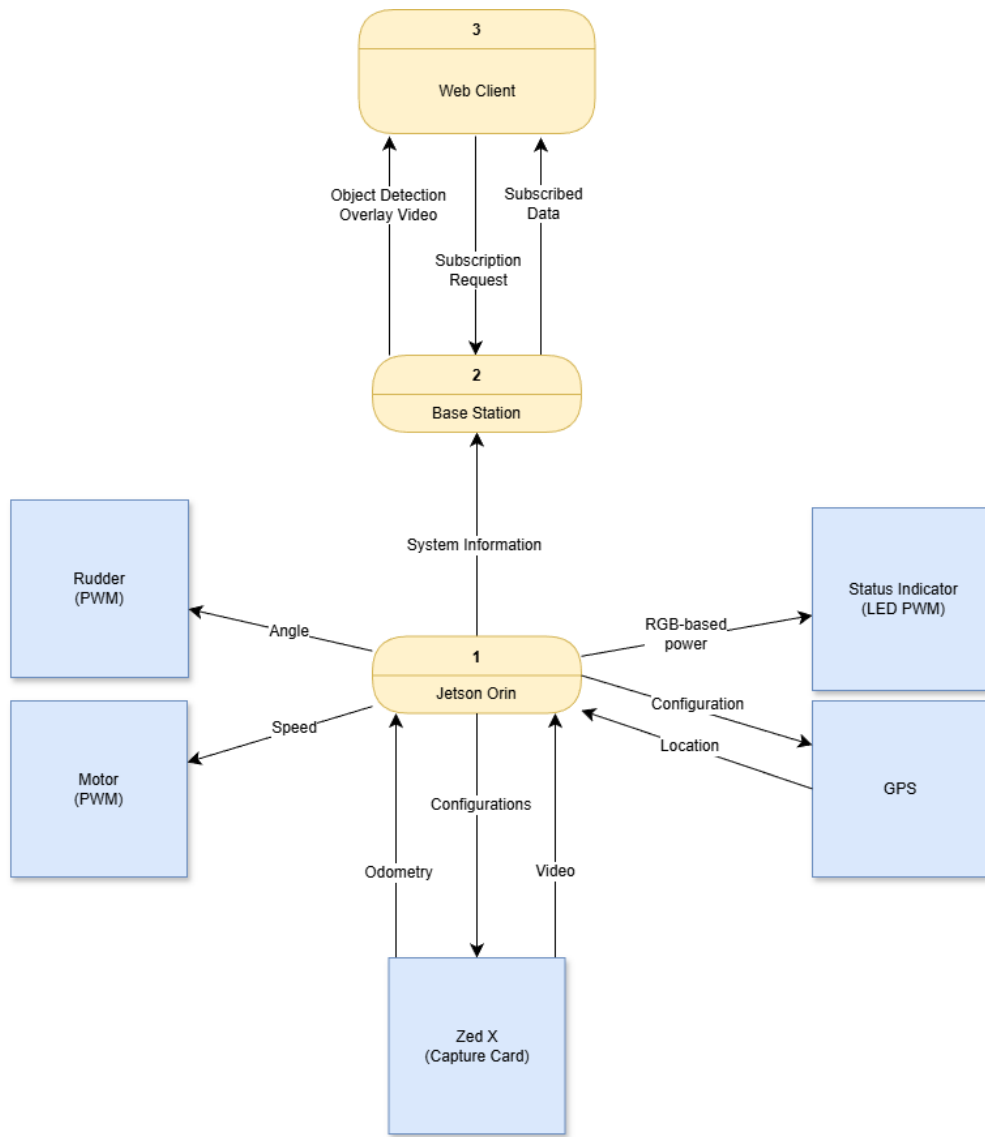


Figure 2: Level 1 Data Flow Diagram

The Loon-E ASV system sends a variety of data which need to be remotely accessed and visualized by users. This data must be transmitted from the ASV to the Base Station, which will then be accessed by users through a web client application.

Primary Computer

The primary computer is responsible for controlling the ASV’s operations, including navigation, sensor data processing, and communication.

Technology	Purpose
ROS2 Humble	Robotics middleware for communication and control
Ubuntu Jazzy Jellyfish	Operating system for the primary computer
Cyclone DDS	(ROS middleware) Chosen data distribution service ¹
Zedx SDK	Software development kit for the Zedx camera
Docker	Containerization platform for software deployment

Table 1: Primary Computer Software Stack

3.1.1 ROS2 Humble

ROS2 Humble is a robotics middleware that provides a framework for building robot applications. It offers a collection of tools, libraries, and conventions that simplify the development of complex robotic systems [6]. We designed a series of nodes—processes that perform specific tasks—to run on the primary computer, which communicate with each other using ROS2 topics and services. Figure 3 illustrates a level 1.1 data flow diagram for the primary computer, which breaks down the primary computer process from Figure 2 into its constituent nodes and their interactions.

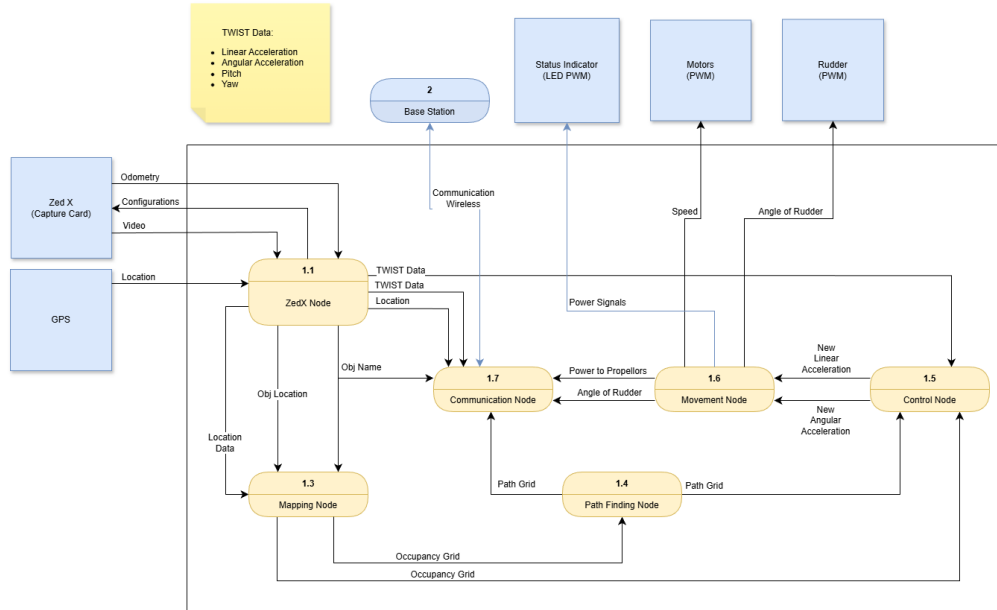


Figure 3: Level 1.1 Data Flow Diagram for Primary Computer

To test and send data from the primary computer to the base station, we set up a ROS2 publisher node that sends simulated sensor data to a ROS2 subscriber node running on the base station. Additionally, for testing purposes, we set up a ROS2 subscriber node on a secondary server computer that runs a simulation on Isaac SIM.

¹ROS2 supports multiple DDS implementations, however, Stereolabs, the manufacturer of the Zedx camera, only provides support for Cyclone DDS [9] [4].

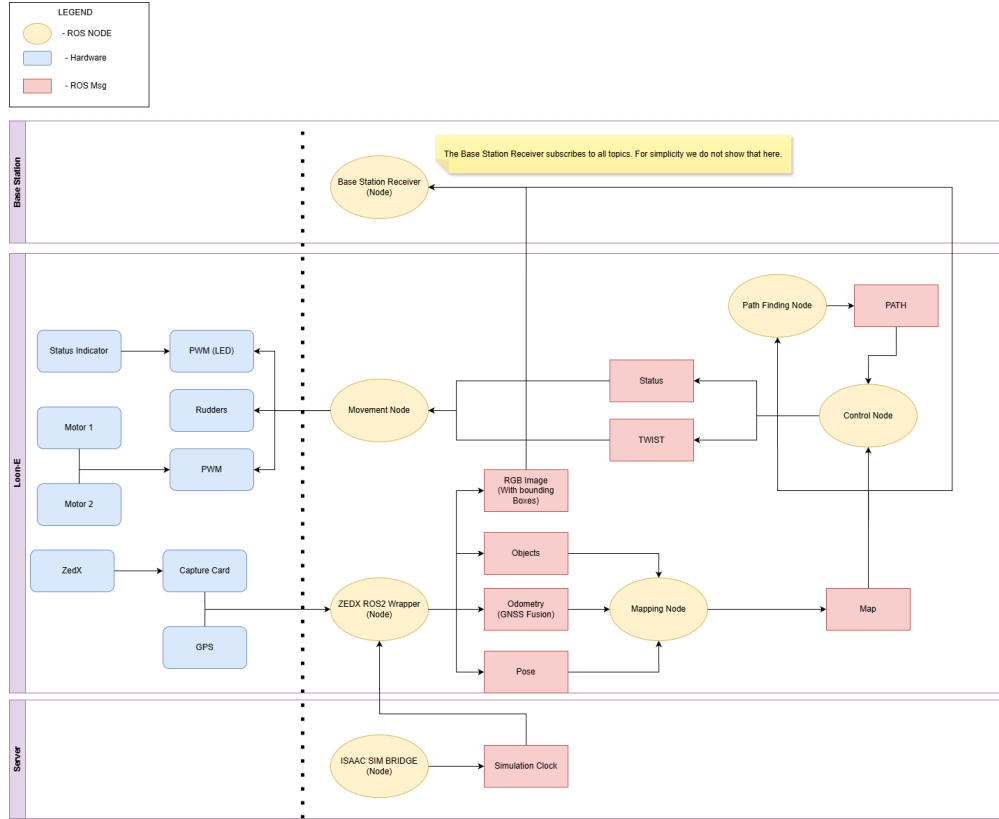


Figure 4: Software Architecture Diagram

The web client provides a user interface for monitoring the ASV remotely. The base station is responsible for receiving data from the ASV for diagnostic monitoring and hosting the web client.

4 Software Development

This section provides information on how to set up the development environment, run the software, and contribute to the Loon-E ASV project. It also includes relevant documentation and resources for developers to effectively contribute to the project.

In order to contribute to the project, developers will need to set up their development environment with the necessary software and tools. The required software and tools for development are listed in Figure 5

- GitHub
- Tailscale
- Rust Desk
- Git
- ROS2 Humble
- Stereolab's ZedSDK 5.3
- Putty (optional, and for Windows only)
- FileZilla (optional, and for Windows only)
- Python 3.10
- Discord (Used for communications)
- LiveTeX (distribution of LaTeX)
- NPM (may change later)
- Microsoft Teams (Primary Document Sharing)
- Docker
- Any preferred LaTeX editor
- Any Preferred IDE
- Any Standard Preferred Web Browser

Figure 5: Software and tools required for development.

Development Resources

A few primary resources are available for you to get started with development:

- documentation
- codebase access
- communication channels

At bare minimum, you'll require Latex for in-depth documentation (I.E this documentation), an IDE for code development, and a web browser for accessing the codebase and communication channels. Further details on what tools you need are in Figure 5.

4.1.1 Documentation

Documentation is split into various channels, each serving a different purpose.

This Document This public document serves as the primary source of information for the Loon-E ASV project, providing comprehensive details on the design, operation, and maintenance of the ASV. It is intended to be a valuable resource for stakeholders and developers involved in the project. It is designed to be modular and easily navigable, allowing readers to quickly find the information they need and so multiple developers can work on different sections without conflicts.

GitHub Wiki The public GitHub Wiki serves as a collaborative platform for developers to share information, best practices, and updates related to the project. It is a living document that evolves with the project and provides a space for developers to contribute their knowledge and insights.

GitHub README The public GitHub README file provides an overview of the project, including its purpose, features, and instructions for getting started. It serves as an introduction to the project for new developers and provides essential information for understanding the project's goals and structure.

GitHub Issues and Pull Requests GitHub Issues and Pull Requests serve as a platform for developers to report bugs, suggest features, and contribute code to the project. They provide a structured way for developers to collaborate and track the progress of the project.

GitHub Project Board The GitHub Project Board is used only for Software related project management and task tracking. It provides a visual representation of the project's progress, allowing developers to see what tasks are in progress, what tasks are completed, and what tasks are upcoming. Debate exists on whether this or Microsoft Teams Kanban board should be the primary project management tool, but for now, the GitHub Project Board is used for software-related tasks, while the Microsoft Teams Kanban board is used for everything else.

Teams Onedrive The Teams Onedrive serves as a centralized repository for project-related documents, including design files, meeting notes, and other resources. It provides a secure and organized space for storing and sharing project materials among team members. You'll need to be added to the Teams Onedrive by a project manager to access it, so reach out to one if you need access (see discord).

4.1.2 Codebase

GitHub Repository The GitHub repository serves as the primary codebase for the Loon-E ASV project, hosting all source code, and some related resources; however, it is recommended to use this document for a single source of truth. It provides a collaborative platform for developers to contribute code, track changes, and manage the project's development.

Branching Strategy No current branching strategy exists, but we are in the process of implementing one. The proposed branching strategy includes a main branch for stable releases, a development branch for ongoing work. This strategy helps to organize the codebase and facilitate collaboration among developers.

Code Reviews Code reviews are conducted through GitHub Pull Requests, allowing team members to review and provide feedback on proposed changes before they are merged into the main codebase. This process helps to maintain code quality and ensure that changes align with the project's goals and standards.

Pull Request Process The pull request process involves creating a new branch for the proposed changes, making the necessary code modifications, and then submitting a pull request for review. Team members can comment on the pull request, suggest improvements, and ultimately approve or request changes before the code is merged into the main branch. It is important to follow the established pull request process to ensure that contributions are properly reviewed and integrated into the project.

4.1.3 Communication Channels

Discord The private Discord server serves as the primary communication channel for the Loon-E ASV project, allowing team members to collaborate, share updates, and discuss project-related topics in real time. It provides a platform for developers to ask questions, share ideas, and stay connected with the rest of the team. You'll need to be added to the Discord server by a project manager to access it, so reach out to our email.

Email Email is used for formal communications and is not for development discussions. It is used mostly for communicating with external stakeholders; however, if you wish to join us then you can reach out to our email and we will get back to you as soon as possible. Our email is: mechatronicsclub@humber.ca

Microsoft Teams Microsoft Teams is used for document sharing and collaboration among team members. It provides a centralized space for storing and sharing project-related documents, including design files, meeting notes, and other resources. It also tracks the team Kanban board, which is used for project management and task tracking. You'll need to be added to the Microsoft Teams by a project manager to access it, so reach out to one if you need access (see discord).

Contribution Guidelines

Refer to each github repository for specific contribution guidelines, but in general, contributions to the project should follow these guidelines:

- Be respectful and collaborative in all interactions with other team members.
- Follow the established coding standards and best practices for the project.
- Use the pull request process for all code contributions, and ensure that your code is properly reviewed before it is merged into the main codebase.
- Report any issues or bugs you encounter in the GitHub Issues, and provide as much detail as possible to help others understand and address the issue.
- Stay engaged with the project and communicate regularly with other team members to ensure that your contributions align with the project's goals and priorities.

4.2.1 Code of Conduct

To maintain a positive and inclusive environment for all contributors, we have established a code of conduct that all team members are expected to follow; See Figure 6 for details.

- Respect and Kindness: Treat others with respect and kindness. Disagreements are fine, but personal attacks, harassment, or any form of disrespectful behavior will not be tolerated.
- English Only: To maintain a cohesive and inclusive community, please keep all communications in English.
- No NSFW/Extreme Content: This is a robotics server focused on learning and collaboration. Any content or discussions deemed NSFW (Not Safe For Work) or extreme in nature are strictly prohibited.
- Unsolicited DMs: Respect the privacy of others. Do not send unsolicited direct messages to members without their consent. If you wish to reach out to someone, please do so in a respectful manner and only after obtaining permission in the server channels.

Figure 6: Code of Conduct

5 Remote Connections

Local development is done on the ASV's primary computer, which is a Jetson Orin. Remote access to the primary computer is necessary for monitoring the ASV's operations and for development purposes.

In order to connect, the lovely folks at TailScale ² have set up a secure VPN connection to the ASV's primary computer, allowing developers to access the system remotely from their own devices. This VPN connection ensures that developers can securely access the ASV's primary computer from anywhere, enabling them to monitor the ASV's operations, access logs, and perform development tasks without needing to be physically present at the ASV's location. At cost, local development on a small device is not scalable for larger teams, fortunately, our software team is relatively small and the VPN connection provides a convenient solution for remote access to the ASV's primary computer.

TailScale Connection

To connect to the ASV's primary computer using TailScale, developers need to follow these steps:

1. Install the TailScale client on their local device (available for Windows, macOS, Linux, iOS, and Android).
2. Create a Github account (if they don't have one already)
3. Notify the software team lead of their Github username so that they can be added to our Github organization.
4. Sign in to TailScale using their Github credentials.³
5. Once signed in, they will see the ASV's primary computer listed as a device in their TailScale dashboard.
6. Click on the ASV's primary computer (named "ubuntu") to establish a secure VPN connection.
7. Once connected, developers can access the ASV's primary computer as if they were on the same local network, allowing them to monitor and develop the software remotely.

RustDesk Connection

In addition to the TailScale VPN connection, developers can also use RustDesk for remote desktop access to the ASV's primary computer. RustDesk is an open-source remote desktop software that allows users to access and control a computer remotely. To use RustDesk for remote desktop access, developers need to follow these steps:

1. Install the RustDesk client on their local device (available for Windows, macOS, Linux, iOS, and Android).
2. Obtain the RustDesk ID and password for the ASV's primary computer from the software team lead.
3. Open the RustDesk client and enter the ASV's primary computer's RustDesk ID and password to establish a remote desktop connection.

²TailScale is a popular VPN solution that provides secure and easy-to-use remote access to devices.

³TailScale let us use their service so long as we are open source and connected to the Github organization. Developers will need to sign in with their Github account to access the VPN.

6 Running the Software

Refer to Section 5 for instructions on how to remotely connect to the ASV’s primary computer; this is needed for running software on the ASV.

Once connected to the primary computer, a custom script has been created to simplify the process of running the software. To start the software, developers can run the following command in the terminal:

```
LoonE --start
```

This command may also freeze the terminal and cause a temporary 10 minute loss of connection to the ASV, but this is normal as the software is starting up and may take some time to initialize. This command will build and execute a docker container from configurations installed from the custom script’s install features.

⁴ It is not recommended to stop the process of building the container.

WARNING

Changes in container will not be saved

You may need to enter the container in multiple terminals to run different processes. Follow the instructions below to enter the container:

```
docker ps # to get the container id or name
docker exec -it <container_id_or_name> /bin/bash
```

For our environment it is set to build ZedXWrapper from our main repository instead of stereolabs’ repository. This is because stereolabs’ repository is actively maintained and may introduce breaking changes that can disrupt our development process. By building from our repository, we have more control over the software and can ensure that it remains stable and compatible with our system, allowing for a smoother development experience. Additionally, it makes it easier for us to set the configuration for the ZedX camera, which is crucial for our ASV’s operations.

To run implemented features within the container, the current command is as follows:

```
ros2 launch loon_e_coms coms_launch.py
```

This command will launch the ROS2 nodes—including ZedX-Wrapper—for the Loon-E ASV’s communication system, allowing the ASV to send and receive data as part of its operations. See the launch file for more details on what processes are launched and how they are configured.

Troubleshooting

If you encounter issues while running the software, here are some common troubleshooting steps:

Camera not connecting If the ZedX camera is not connecting or functioning properly, try the following steps:

- Ensure that the ZedX camera is properly connected to the primary computer and powered on.
- Check the camera’s physical connection and power status, and ensure that it is securely plugged in.
- Verify that the necessary drivers and software for the ZedX camera are installed and up to date

⁴A docker container either downloads the scripts or establishes a volume from the host which has the scripts.

```
apt list --installed | grep zed
```

Should show the zedlink-duo and ros-humble-zed-msgs packages

- Check diagnostics

```
ZED_Diagnostic
```

This should show diagnostic messages from the ZedX camera, which can help identify any issues with the camera's operation. **Note:** If in a container the diagnostic tool will not detect the driver, you will need to run it on the host machine.

ROS2 nodes not communicating If the ROS2 nodes are not communicating properly, try the following steps:

- Ensure that all ROS2 nodes are properly launched and running.
- Check the ROS2 topic list to verify that the expected topics are being published and subscribed to:

```
ros2 topic list
```

- Use ROS2 topic echo to check the data being published on specific topics:

```
ros2 topic echo <topic_name>
```

- Check for any error messages in the terminal where the nodes are running, which may indicate issues with node communication or configuration.
- Verify that the ROS2 middleware (Cyclone DDS) is properly configured and running, as it is responsible for facilitating communication between nodes.
- Ensure the topics are being run on the same Domain ID, as ROS2 uses DDS which requires all nodes to be on the same Domain ID to communicate.
- Check for any firewall or network issues that may be preventing communication between nodes, especially if running across multiple machines.
- Verify that the ROS2 environment variables are properly set, as incorrect environment configurations can lead to communication issues between nodes.
- If using Docker, ensure that the container has the necessary network configurations to allow communication between nodes, especially if nodes are running in different containers or on the host machine.

7 ROS2 Design

Node Name	Purpose
Control Node	Executes control logic and navigation commands
Map Node	Maintains and updates local environmental map
Movement Node	Manages motor control and vehicle movement
Planning Node	Computes navigation paths to destination
Base Station Node	Monitors and logs all system data
ROS2 Publish Clock Node	Synchronizes simulation timing across nodes
Zedx Camera Node	Processes stereo vision and object detection

Table 2: Primary Computer ROS2 Nodes

Node Name	Subscriptions	Publications
Control	map (local), objects, locations	PWM
Map	obstacles, locations	map (local)
Movement Node	PWM	not implemented
Base Station	ALL	None
Planning node	map (local), destination	path
ROS2 Publish Clock	None	clock
Zedx Camera	clock ⁵	objects, pose, image_rect_color, map (global)

Table 3: Current Primary Computer ROS2 Nodes Data Distribution

movement needs pose and odometry data to move. zedx camera needs to publish objects and mapping receive it

Node Name	Subscriptions	Publications
Control	map (local), map (global), objects	twist, status
Map	objects ⁶	map (local)
Movement Node	twist, status	health (movement), power
Base Station	ALL	None
ROS2 Publish Clock	None	clock
Zedx Camera	clock	objects, pose, image_rect_color, map (global)

Table 4: Proposed Computer ROS2 Node Data Distribution

⁵The Zedx Camera node subscribes to the clock topic to synchronize its data publishing with Isaac SIM.[10]

⁶Zedx Objects contains location and classification data [8].

A Glossary

Autonomous Surface Vehicle (ASV) An autonomous surface vehicle (ASV) is an unmanned vessel that operates on the sea surface without real-time input or control from human operators.[12]

Base Station A fixed radio communications station that can communicate with mobile radio devices in its neighbourhood. Forms of radio communication that have base stations include radio wireless data and mobile telephony...[1]

Loon-E Humber ASV's current ASV.[**HuMBERASVTEAM**]

HumberASV Humber Polytechnic's Mechatronics club. A multidisciplinary team of passionate Humber Polytechnic students and professionals dedicated to advancing autonomous maritime technology.[2]

LIDAR LiDAR, an acronym for "light detection and ranging," is a remote-sensing technology that uses laser beams to measure precise distances and movement in an environment, in real time.[3]

Robot Operating System (ROS) The Robot Operating System (ROS) is a set of software libraries and tools that help you build robot applications.[5]

B Bibliography

- [1] Andrew Butterfield and John Szymanski. *base station*. 2018. DOI: 10.1093/acref/9780198725725.013.0348. URL: <https://www.oxfordreference.com/view/10.1093/acref/9780198725725.001.0001/acref-9780198725725-e-348>.
- [2] HumberASV. *Our Team*. 2026. URL: <https://humberasv.ca/team> (visited on 03/17/2026).
- [3] IBM. *What is LIDAR?* 2026. URL: <https://www.ibm.com/think/topics/lidar> (visited on 03/17/2026).
- [4] Open Robotics. *DDS implementations*. 2026. URL: <https://docs.ros.org/en/humble/Installation/RMW-Implementations/DDS-Implementations.html> (visited on 04/16/2026).
- [5] Open Robotics. *ROS*. 2026. URL: <https://www.ros.org/> (visited on 03/17/2026).
- [6] Open Robotics. *ROS2 Documentation: Humble*. 2026. URL: <https://docs.ros.org/en/humble/index.html> (visited on 04/16/2026).
- [7] Amelia Soon et al. *RoboBoat 2026 Technical Design Report – Humber ASV Loon-E*. Toronto, ON, Canada, 2026. URL: <https://humberasv.ca/docs> (visited on 04/16/2026).
- [8] STEREO LABS. *Custom Object Detection and Tracking with ROS 2*. 2026. URL: <https://www.stereolabs.com/docs/ros2/custom-object-detection> (visited on 04/16/2026).
- [9] STEREO LABS. *DDS Middleware and Network tuning*. 2026. URL: <https://www.stereolabs.com/docs/ros2/dds-and-network-tuning/> (visited on 04/16/2026).
- [10] STEREO LABS. *Using ZED with ROS 2 and Isaac Sim*. 2026. URL: https://www.stereolabs.com/docs/isaac-sim/ros2_integration (visited on 04/16/2026).
- [11] STEREO LABS. *Zed SDK Documentation*. 2026. URL: <https://www.stereolabs.com/docs/> (visited on 04/16/2026).
- [12] Arthur Trembanis, Mark Lundine, and Kaitlyn McPherran. “Coastal Mapping and Monitoring”. In: *Encyclopedia of Geology (Second Edition)*. Ed. by David Alderton and Scott A. Elias. Second Edition. Oxford: Academic Press, 2021, pp. 251–266. ISBN: 978-0-08-102909-1. DOI: <https://doi.org/10.1016/B978-0-12-409548-9.12466-2>. URL: <https://www.sciencedirect.com/science/article/pii/B9780124095489124662>.